Reporte Técnico de Actividades Práctico-Experimentales Nro. 004

Análisis comparativo de herramientas de simulación mediante un modelo común

1. Datos de identificación del estudiante y la práctica

Estudiante	Richard Ariel Chamba Santin
Asignatura	Simulación
Ciclo y unidad	Quinto ciclo; U1: Introducción a la Simulación
Resultado de aprendizaje	R1. Aplica análisis estadísticos de los datos de la simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica	Nro. 004
Docente	José O. Guamán Q.
Fechas indicadas en la guía	22 y 23 de octubre de 2026
Lugar y tiempo planificado	Aula; 3 horas

2. Objetivos de la práctica

El objetivo general de la guía es comparar lenguajes y plataformas para implementar modelos de simulación. Sus objetivos específicos son:

• Implementar un modelo sencillo en distintas herramientas.

• Comparar su sintaxis, facilidad de implementación y visualización.

• Identificar el ámbito de aplicación de cada herramienta.

En este trabajo se implementó el modelo SIR en Java y NetLogo. La ficha también pide Python con SimPy; esa versión no forma parte del proyecto y se identifica como alcance pendiente.

3. Materiales y herramientas

• Computador con Linux.

• JDK 11 o posterior y NetLogo 7.0.4.

• Editor de código y archivos CSV.

Java se organiza en Modelo, Vista y Controlador. NetLogo se utiliza para la simulación basada en agentes, la importación del CSV generado por Java y la representación de las gráficas.

4. Procedimiento y metodología ejecutada

4.1 Fundamento del modelo

El modelo SIR divide la población en susceptibles (S), infectados (I) y recuperados (R), con una población constante: $N = S + I + R$. La guía presenta $dS/dt = -\beta SI/N$, $dI/dt = \beta SI/N - \gamma I$ y $dR/dt = \gamma I$, además del valor de referencia $R_0 = \beta/\gamma = 3$. Para esta actividad se aplicó la variante discreta por agentes indicada en el procedimiento, en lugar de resolver las ecuaciones diferenciales directamente.

Parámetro	Valor
Población total	100
Susceptibles iniciales (S_0)	95
Infectados iniciales (I_0)	5
Recuperados iniciales (R_0)	0
Probabilidad de infección por contacto (β)	0,30
Probabilidad de recuperación por paso (γ)	0,10
Tiempo máximo	100 pasos

Tabla 1. Parámetros iniciales establecidos por la ficha.

4.2 Regla por paso

En cada paso, cada infectado elige al azar una persona diferente de sí mismo. Si esa persona era susceptible al inicio del paso, se intenta el contagio con probabilidad β = 0,30. Cada persona que estaba infectada al comenzar el paso se recupera con probabilidad γ = 0,10. Los eventos se aplican sobre una copia del estado actual y los cambios se realizan simultáneamente, por lo que un recién infectado comienza a participar a partir del paso siguiente.

Un infectado puede contagiar y recuperarse en el mismo paso; cada individuo se cuenta una sola vez. Los contactos de infectados distintos pueden coincidir. Los recuperados conservan su estado. El paciente cero es la persona identificada como 0, una de las cinco infectadas iniciales.

La ficha permite seleccionar contactos cercanos o elegidos al azar. Se usa un contacto aleatorio entre toda la población susceptible en ambas implementaciones. En NetLogo, el movimiento de las figuras es ilustrativo y no determina el contagio.

4.3 Secuencia de ejecución

• Crear 100 personas: 95 susceptibles, 5 infectadas y 0 recuperadas; identificar al paciente cero.

• Registrar el conteo inicial en t = 0.

• Para cada tiempo de 1 a 100, copiar estados, probar contagios y recuperaciones y actualizar los estados juntos.

• Contar S, I y R en cada paso.

• Guardar en Java las 101 filas con encabezado tiempo,susceptibles,infectados,recuperados.

• Abrir el modelo NetLogo, cargar el CSV y comparar las curvas con la simulación propia de NetLogo.

El CSV contiene los valores agregados necesarios para las gráficas; no guarda posiciones ni historiales individuales. En cada tiempo se conserva S + I + R = 100.

4.4 Arquitectura e interacción

Componente	Responsabilidad
Modelo Java	Persona y SimulacionSIR: estados, parámetros, reglas y conteos.
Vista Java	VistaConsola: parámetros y resumen en terminal.
Controlador Java	Coordina el modelo, escribe resultados_java.csv y lo copia a netlogo/.
NetLogo	setup crea personas y carga Java; go continuo ejecuta un paso; las figuras se mueven despacio y las gráficas comparan las series.

Tabla 2. Componentes de la implementación.

Java se ejecuta desde la raíz del proyecto. En NetLogo se pulsa setup y luego go. La pausa visual predeterminada de 0,10 s regula el ritmo observado, no la unidad de tiempo de la simulación. CARGAR

JAVA permite releer el CSV sin reiniciar la corrida de NetLogo.

5. Resultados esperados y alcance

La guía espera modelos SIR en NetLogo, Python/SimPy y Java; CSV de Python y Java; gráficas de S, I y R; visualización de datos importados a NetLogo; comparación de resultados; y análisis de ventajas, limitaciones y diferencias aleatorias. Esta entrega produjo Java y NetLogo, el CSV Java y cinco gráficas. Python/SimPy y su CSV no se implementaron, así que la comparación de tres plataformas está incompleta.

6. Resultados obtenidos

Medida	Java	NetLogo
Máximo de infectados	29	44
Primer tiempo en el máximo	31	11
S al tiempo 100	13	8
I al tiempo 100	0	0
R al tiempo 100	87	92

Tabla 3. Resumen de una ejecución por herramienta, con semilla 42.

Java inicia en 95, 5, 0, alcanza 29 infectados en el paso 31 y finaliza en 13, 0, 87. NetLogo inicia con los mismos conteos, alcanza 44 infectados en el paso 11 y termina en 8, 0, 92.

Los resultados son de una corrida por herramienta, no promedios de réplicas ni intervalos de confianza. Aunque se configuró la misma semilla numérica, Java y NetLogo usan generadores y secuencias aleatorias distintas; la semilla repite cada implementación, pero no sincroniza ambas.

6.1 Gráficas de la simulación

Las cinco gráficas corresponden a las solicitadas en NetLogo: evolución SIR de Java, evolución SIR de NetLogo y tres comparaciones separadas de S, I y R. Las curvas de Java se importan desde el CSV; las de NetLogo se acumulan mientras corre go.

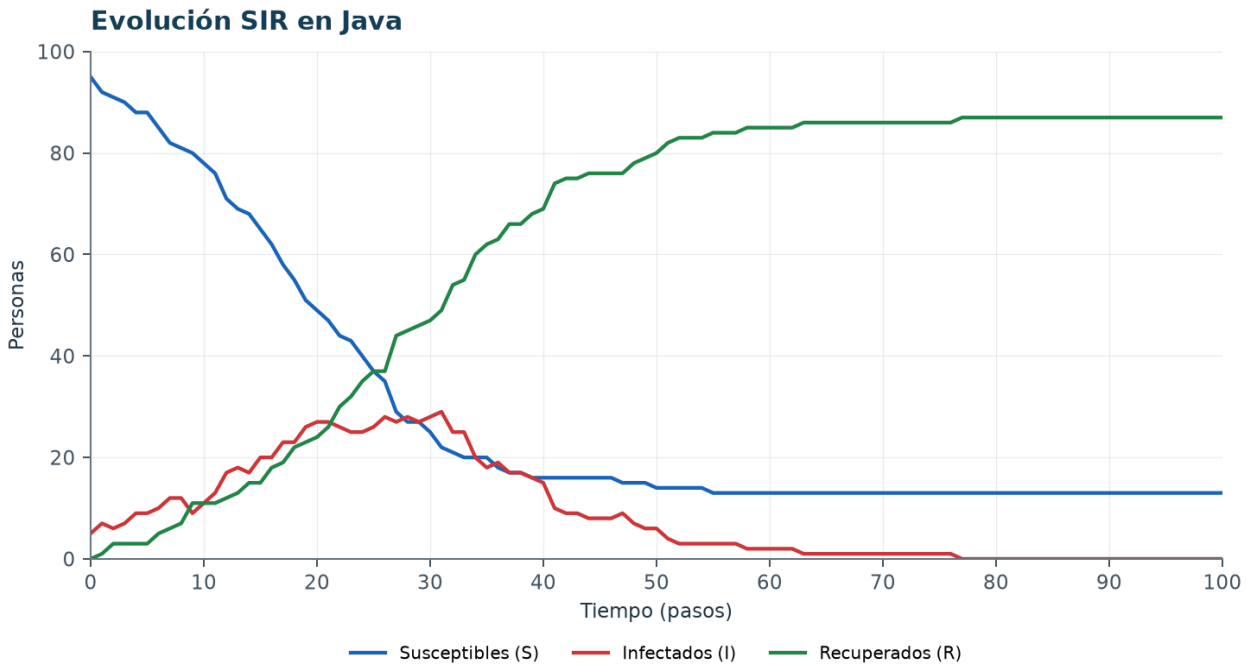


Figura 1. La serie parte de S=95, I=5 y R=0. Los infectados alcanzan 29 en t=31 y llegan a cero en t=77.

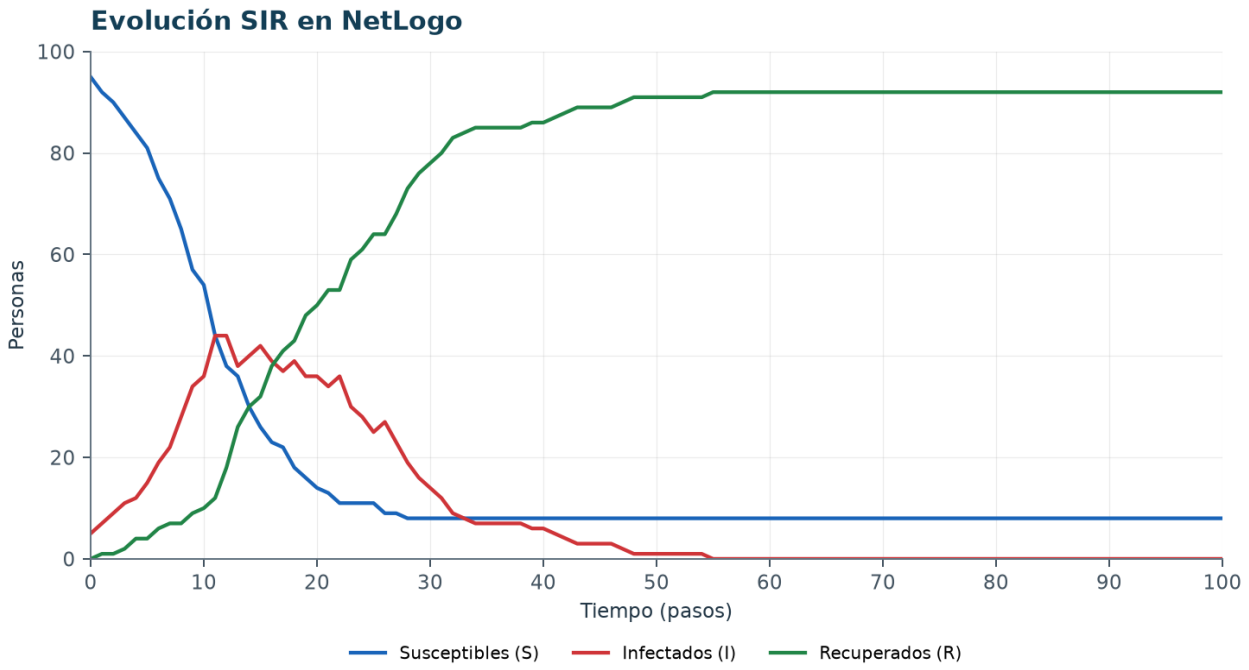


Figura 2. En esta corrida I alcanza 44 en t=11. En t=100 quedan S=8, I=0 y R=92.

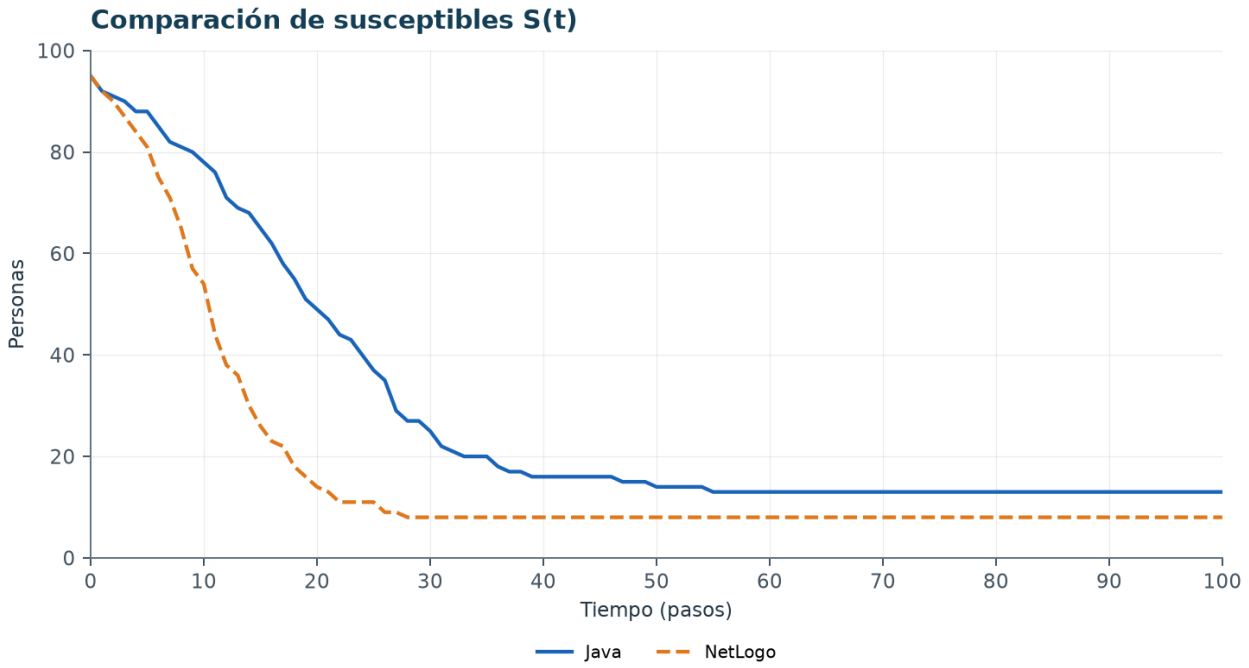


Figura 3. Las dos curvas descienden. En t=100 Java registra 13 susceptibles y NetLogo 8.

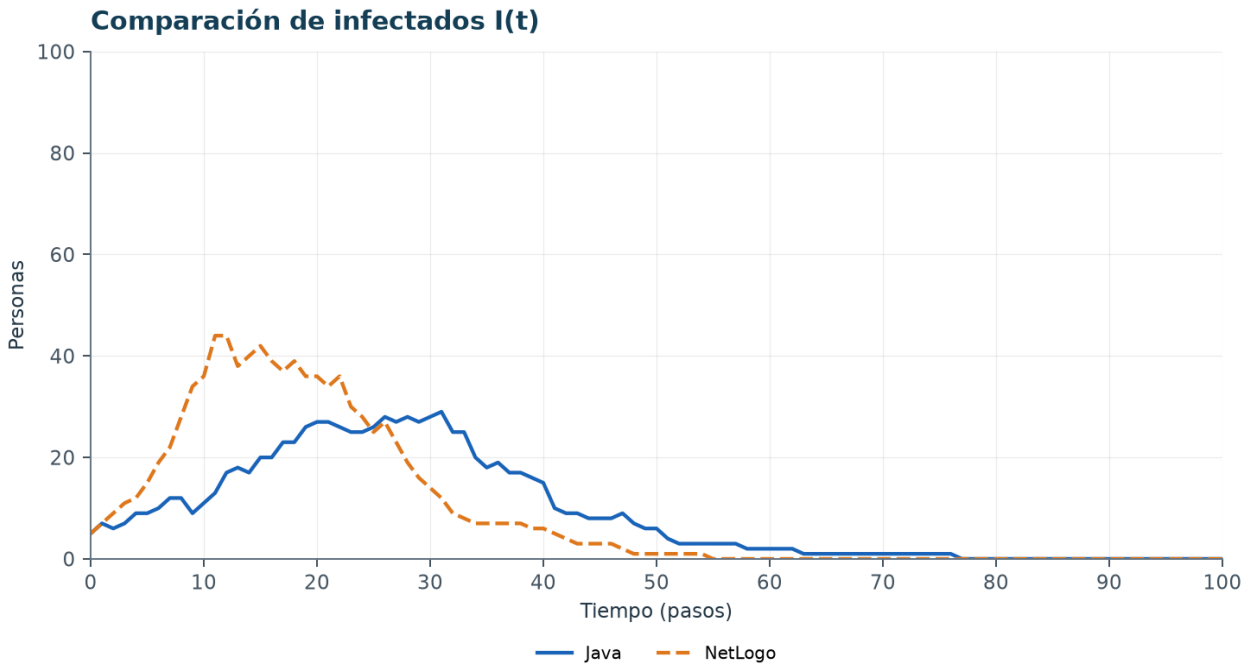


Figura 4. Java alcanza 29 en t=31; NetLogo, 44 en t=11. Ambas series llegan a cero al final.

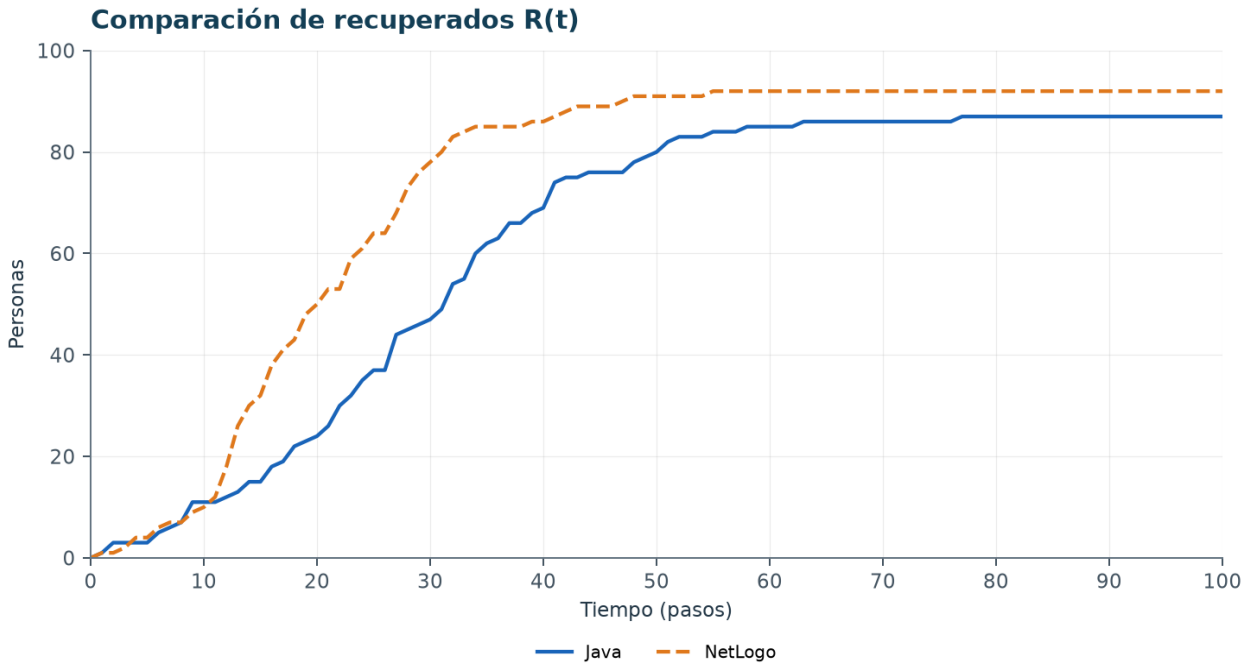


Figura 5. Los recuperados se acumulan: 87 en Java y 92 en NetLogo al paso 100.

7. Análisis de las gráficas

Evolución SIR de Java. S disminuye de 95 a 13 y R crece de 0 a 87. I alcanza 29 en t=31 y luego baja a cero en t=77. La forma concuerda con el modelo sin reinfección: S desciende, I primero sube y luego baja, y R solo aumenta.

Evolución SIR de NetLogo. I llega a 44 en t=11 y cae a cero antes del paso 100; al final hay 8 susceptibles y 92 recuperados. El movimiento permite observar los agentes; la cercanía de las figuras no determina los contactos.

Comparación de susceptibles S(t). Ambas series descienden y finalizan con cero infectados activos; Java termina con 13 susceptibles y NetLogo con 8. La diferencia corresponde a la trayectoria aleatoria de esta

corrida, no a distintos parámetros.

Comparación de infectados I(t). El pico de NetLogo es más alto y temprano: 44 en t=11 frente a 29 en t=31 en Java. Una sola corrida no muestra que una herramienta produzca sistemáticamente más contagios; para esa conclusión se requieren varias semillas y comparar medias y dispersión.

Comparación de recuperados R(t). R crece de manera acumulativa porque los recuperados no vuelven a S o I. Termina en 87 para Java y 92 para NetLogo, a la vez que I llega a cero en ambas.

El análisis es descriptivo y se basa en una sola ejecución de cada programa. Para estimar variabilidad y obtener una comparación estadística más sólida, se deberían repetir las simulaciones con varias semillas y reportar medias y desviaciones.

7.1 Comparación de herramientas

Aspecto	NetLogo	Python/SimPy	Java
Facilidad de implementación	Reglas de agente en el mismo modelo.	No implementado.	Estados y cambios explícitos en clases.
Manejo de agentes	Tortugas nativas y visibles.	No implementado.	Un objeto por persona.
Generación de datos	Serie propia e importación del CSV Java.	No implementado.	CSV agregado por tiempo.
Visualización	Movimiento y gráficas integradas.	No implementado.	Resumen en consola; gráficas en NetLogo.
Flexibilidad	Práctico para observar agentes e interacciones.	No evaluado.	MVC permite separar reglas y coordinación.
Tiempo e integración	Permite inspección visual y comparación de series.	Sin medición.	El CSV facilita intercambio de resultados.

Tabla 4. Evaluación basada en las herramientas efectivamente utilizadas. No se inventan mediciones para Python/SimPy.

8. Preguntas de control

1. ¿Qué representa el modelo SIR?
La evolución de una población expuesta a una infección, distribuida entre susceptibles, infectados y recuperados.

2. ¿Qué representan los estados S, I y R?
S son quienes pueden infectarse; I son quienes tienen la infección y pueden transmitirla; R son quienes se recuperaron y no vuelven a infectarse en este modelo.

3. ¿Qué función cumplen los parámetros β y γ?
β = 0,30 es la probabilidad de contagio por contacto con un susceptible; γ = 0,10 es la probabilidad de recuperación por paso.

4. ¿Por qué es importante usar los mismos parámetros en las tres herramientas?
Para atribuir las diferencias a la implementación o al azar, no a poblaciones, tasas o tiempos configurados de otra forma. Esta entrega compara Java y NetLogo; no implementa Python.

5. ¿Por qué Python, Java y NetLogo pueden producir resultados diferentes utilizando el mismo modelo?
Sus generadores pueden producir secuencias aleatorias distintas y su orden de actualización también puede afectar una trayectoria. Repetir parámetros no obliga a que coincida una corrida aleatoria.

6. ¿Qué ventajas ofrece NetLogo para representar sistemas basados en agentes?
Incluye agentes y mundo visual, facilita observar estados y movimiento, y relaciona reglas individuales con conteos agregados.

7. ¿Qué ventajas ofrece SimPy para desarrollar simulaciones basadas en eventos?
Permite expresar procesos que esperan eventos y administra el calendario de eventos discretos. SimPy no se implementó en este trabajo.

8. ¿Qué ventajas ofrece Java para implementar modelos de simulación?
Permite organizar el modelo en clases, representar estados de manera explícita y controlar la lógica y la escritura de resultados.

9. ¿Qué función cumple el CSV en la integración Python/Java NetLogo?
Transporta tiempo, S, I y R para que NetLogo grafique datos generados en otra herramienta. En esta práctica se importan los datos de Java.

10. ¿Qué ventajas presenta separar la generación de datos de la visualización?
Permite repetir o cambiar el modelo sin reescribir las gráficas, reutilizar datos y comprobar lo que se transmite mediante un formato sencillo.

9. Conclusiones

• Java implementó el SIR discreto con los valores de la ficha, organización MVC y salida CSV para NetLogo.

• NetLogo representó agentes en movimiento y permitió graficar la serie Java y su propia simulación en cinco vistas.

• Los picos de una corrida fueron distintos entre Java y NetLogo, lo que ilustra el efecto de los generadores aleatorios y motiva repetir corridas antes de generalizar.

• El objetivo de comparación se cubrió parcialmente: para completar la comparación solicitada en la guía falta implementar Python/SimPy y añadir sus datos.

10. Declaración de uso de inteligencia artificial

Se utilizó ChatGPT/Codex como apoyo para organizar el borrador del informe, revisar los requisitos de la ficha y proponer redacciones iniciales de la metodología, el análisis y las respuestas de control. Se redactó un prompt general representativo para describir esta ayuda, no como transcripción literal de toda la conversación. Los resultados numéricos se tomaron de las ejecuciones del programa y se comprobaron contra la conservación de la población. La revisión final, la comprensión y la responsabilidad académica corresponden al estudiante.

Prompt general representativo

> Revisa la guía de la práctica y el proyecto existente de simulación SIR. Conserva la organización y el formato usados en las prácticas anteriores. Redacta un borrador de metodología, resultados, análisis de gráficas y respuestas a las preguntas de control, basándote en los parámetros, el código y los datos realmente disponibles. Explica S, I, R, beta y gamma; describe la simulación discreta por agentes y la integración CSV de Java con NetLogo. Distingue los resultados ejecutados de los requisitos pendientes, identifica las limitaciones del análisis aleatorio y no inventes cifras ni evidencias. Incluye una declaración transparente de uso de IA.

11. Bibliografía

Guamán Q., J. O. (2026). Guía de Actividades Práctico-Experimentales Nro. 004: Análisis comparativo de herramientas de simulación mediante un modelo común. FEIRNNR, Carrera de Computación.

Wilensky, U. (1999). NetLogo. Center for Connected Learning and Computer-Based Modeling, Northwestern University. https://www.netlogo.org/

Oracle. (2024). Java Platform, Standard Edition Documentation. https://docs.oracle.com/en/java/